

SudoOS-Plus

双架构 Rust 操作系统设计方案

RISC-V 64 / LoongArch 64

OSCOMP 2026 决赛技术报告

代码基线 5f82c46888e417717314b1ed4d9296072cef3170

评审稿 2026-08-12



目录

1. 项目概述	5
2. 设计目标与约束	5
2.1 设计目标	5
2.2 工程约束	5
2.3 当前边界	6
3. 工程结构	6
4. 双架构启动与公共初始化	7
4.1 统一入口	7
4.2 RISC-V 64 启动	7
4.3 LoongArch 64 启动	7
4.4 kernel_main 初始化次序	7
5. 内存管理设计	8
5.1 物理内存	8
5.2 内核堆	8
5.3 页表与地址空间	8
5.4 VMA	9
5.5 缺页处理	9
5.6 MAP_FIXED 原子替换	9
5.7 共享文件映射写回	9
6. SMP、调度与并发控制	10
6.1 CPU 生命周期	10
6.2 调度器	10
6.3 MM 切换	10
6.4 IPI 与 TLB shutdown	11
6.5 锁与锁序	11
7. 进程、线程与退出语义	11
7.1 所有权模型	11
7.2 clone 与 execve	11
7.3 退出与等待	11
7.4 futex 与 robust list	12
8. Linux ABI 与用户态兼容	12
8.1 统一 syscall ABI	12
8.2 系统调用覆盖	12

8.3 ELF 与动态装载	13
9. VFS、文件系统与设备	13
9.1 VFS	13
9.2 ext4 overlay	14
9.3 pipe、TTY 与 PTY	14
9.4 procfs 与 sysfs	14
9.5 VirtIO 与块层	14
9.6 网络与随机数	14
10. 决赛执行器	15
10.1 测试模式选择	15
10.2 镜像适配原则	15
11. CAgent 实现与验证	15
11.1 运行准备	15
11.2 双架构结果	16
12. BuildStorm 实现	16
12.1 负载阶段	16
12.2 热点 target 路由	16
12.3 工具链兼容	17
12.4 长时间运行保障	17
13. BuildStorm 问题定位与修复过程	17
13.1 第一阶段：从环境分到真实编译	17
13.2 第二阶段：多线程退出与等待	17
13.3 第三阶段：VMA 与固定映射	17
13.4 第四阶段：产物路径	17
13.5 第五阶段：正确大小的全零 ELF	18
14. BuildStorm 双架构结果	18
15. 构建、测试与质量门禁	18
15.1 提交构建	18
15.2 本地专项回归	19
15.3 成功判据	19
15.4 脚本一致性	19
16. 正确性约束清单	19
17. 已知风险与后续工作	20
17.1 镜像差异	20
17.2 性能	20

17.3 可维护性	20
17.4 长期能力	20
18. 复现步骤	21
18.1 环境	21
18.2 构建	21
18.3 CAgent	21
18.4 BuildStorm	21
19. 文件导航	22
20. 结论	22
附录 A：本地验证记录	23
A.1 RISC-V BuildStorm	23
A.2 LoongArch BuildStorm	23
A.3 CAgent	23
附录 B：参考资料	23

1. 项目概述

SudoOS-Plus 是一个以 Rust `no_std` 为主体、面向 QEMU virt 平台的双架构操作系统内核。项目从启动、页表与物理内存管理起步，逐步建立了 SMP、抢占式调度、进程与线程、Linux asm-generic 64 位系统调用、ELF 动态装载、VFS、ext4、VirtIO、网络和比赛评测执行链。RISC-V 64 与 LoongArch 64 共用绝大多数内核策略，架构目录只保留启动、异常入口、时钟、中断、页表机制和 SMP 唤醒等硬件相关实现。

本阶段的主要任务不是增加孤立的系统调用数量，而是让真实的复杂用户态程序在内核上形成闭环：CAgent 需要动态链接的 shell、进程并发、时间、网络状态和文件系统操作；BuildStorm 则需要 Cargo、rustc、链接器和数百个 crate 在八核、8 GiB 环境中长时间稳定运行。后者会同时施压地址空间、页表、文件映射、线程退出、futex、信号、epoll、VFS 一致性和块设备写入路径。

截至本文对应版本，双架构已完成以下本地验证：

- `make all` 同时生成 `kernel-rv` 与 `kernel-la`；
- RISC-V 与 LoongArch 的 CAgent 十项测试均输出 `pass`，官方脚本正常退出；
- RISC-V BuildStorm 生成 1,685,224 字节的有效 ELF，输出 `ok=true`；
- LoongArch BuildStorm 生成 1,714,736 字节的有效 ELF，输出 `ok=true`；
- 两个 BuildStorm 产物的内核侧与用户侧文件头均为 `7f454c46`；
- 最终评分仍以平台镜像、平台基线和平台 judge 的实际运行结果为准。

2. 设计目标与约束

2.1 设计目标

- 1 双架构共享策略。内存管理、调度、进程、VFS、ELF 与系统调用尽量共用，避免两套端口随功能增长产生语义漂移。
- 2 Linux 用户态兼容。采用 asm-generic 64 位系统调用 ABI，优先保证 glibc、BusyBox、Cargo 和 rustc 的真实调用链。
- 3 正确性优先。所有评分标志由官方脚本根据真实返回值和产物检查生成，不在内核中伪造成功输出。
- 4 SMP 下的资源闭环。用户地址空间、TLB、线程退出、fd 回收和等待唤醒必须在多核并发下保持严格次序。
- 5 离线可复现构建。提交入口固定为 `make all`，Cargo 依赖置于 `vendor/cargo`，构建过程不依赖网络。
- 6 诊断可追溯。长时间负载需要阶段标志、超时边界和失败上下文，便于区分内核故障、镜像差异与评测环境波动。

2.2 工程约束

- 内核使用 Rust 2024 Edition，提交构建工具链为 `nightly-2025-01-18`；
- `panic = "abort"`，不依赖栈展开；
- 当前主要验证对象为 QEMU 的 RISC-V virt 与 LoongArch virt；
- 决赛 BuildStorm 环境按 8 vCPU、8 GiB 内存运行；
- 官方磁盘镜像为完整 Debian/glibc 用户态，并包含 Rust 工具链、TGOSKits 源码与离线缓存；
- ext4 磁盘承担基础镜像和持久路径，热点编译目录可路由到 tmpfs；

- 用户态程序可能长期运行并创建大量线程、映射与短生命周期文件，不能使用只适合 smoke test 的固定小容量结构。

2.3 当前边界

项目已覆盖决赛负载所需的主链，但不将自己描述为完整 Linux 实现。以下能力仍有明确边界：

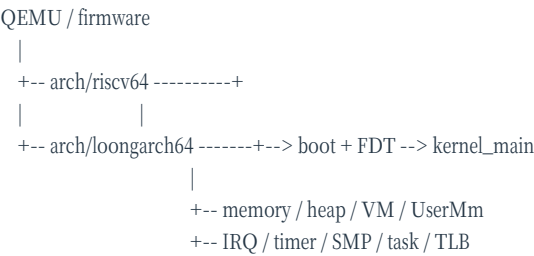
- signal restart、job control 和部分边缘 flag 未完全覆盖 Linux 全语义；
- ext4 写入层以评测负载所需行为为重点，不等同于完整 journal 与崩溃恢复实现；
- 网络主要服务于基础 socket 兼容与回环/状态查询，并非完整高性能网络栈；
- fork 当前侧重正确性，仍可继续引入按需复制和更细粒度页缓存优化；
- 实体机驱动、NUMA、IOMMU 和安全隔离不在本阶段范围内。

3. 工程结构

Cargo workspace 由九个成员组成。

Crate	目录	职责
myos-kernel	kernel/	公共内核入口、调度、进程、系统调用、VFS、设备和比赛执行器
arch-riscv64	arch/riscv64/	RISC-V 启动、SBI、Sv39、trap、时钟与 SMP
arch-loongarch64	arch/loongarch64/	LoongArch 启动、DMW、页表、trap、时钟与 SMP
myos-boot	boot/	跨架构启动参数和 BootInfo
myos-fdt	firmware/fdt/	FDT 校验、遍历与硬件资源枚举
myos-mm	mm/	地址类型、VMA、页表、buddy、slab、ASID 与 TLB 请求
myos-runtime	runtime/	早期控制台和格式化输出适配
myos-sync	sync/	最底层自旋锁原语
myos-vfs	vfs/	VFS 类型、文件操作、fd table、flags 与 errno

核心 Rust 代码约 57,000 行。[kernel/src/user.rs](#) 集中承载用户态兼容路径，体量较大；与之配套的 [process.rs](#)、[user_mm.rs](#)、[syscall.rs](#)、[fs/](#) 和 [task/](#) 分别约束所有权、地址空间、ABI、文件系统和调度语义。



```

+-- Process / Thread / signal / syscall
+-- ELF / exec / dynamic loader
+-- VFS / tmpfs / ext4 / procfs / sysfs
+-- VirtIO block / net / RNG
+-- OSCOMP final runner

```

4. 双架构启动与公共初始化

4.1 统一入口

两个架构最终都进入：

```
pub extern "C" fn rust_entry(arg0: usize, arg1: usize, arg2: usize) -> !
```

架构层先把固件或 direct-boot 参数转换为公共 `BootInfo`，再交给 `kernel_main`。公共代码只通过 `crate::arch` 使用架构能力；若目标不是 RISC-V 64 或 LoongArch 64，构建会在编译期失败。

4.2 RISC-V 64 启动

- 1 OpenSBI 将控制权交给低地址入口；
- 2 启动汇编建立临时 Sv39 页表；
- 3 写入 `satp` 并执行 `sfence.vma`；
- 4 跳入高半内核映射；
- 5 初始化 `gp`、启动栈和 BSS；
- 6 保存 hart ID 与 FDT 地址；
- 7 进入公共 Rust 入口；
- 8 secondary hart 通过 SBI HSM 唤醒并进入独立栈。

RISC-V 用户态 trap 入口必须先恢复内核拥有的 `tp`，再访问 per-CPU 状态。用户态 TLS 的 `tp` 被保存在 trap frame 中，`sscratch` 指向任务栈锚点；返回用户态前重新构造锚点，保证线程迁移后仍能在正确 CPU 上恢复。

4.3 LoongArch 64 启动

- 1 QEMU direct boot 跳入物理入口；
- 2 设置 cached/uncached DMW；
- 3 开启分页并切换到内核虚拟地址；
- 4 解析启动参数和 FDT；
- 5 建立最终页表、TLB refill 和异常入口；
- 6 通过架构 SMP 机制启动 secondary CPU；
- 7 进入与 RISC-V 相同的公共初始化序列。

LoongArch 的 direct boot 命令行由物理内存中的有界 NUL 字符串读取，可直接识别 `sudoos.oscomp=...` 模式；该路径避免依赖尚未建立的通用 VFS。

4.4 kernel_main 初始化次序

初始化严格遵守依赖关系：

- 1 校验 FDT blob，枚举 RAM、CPU、VirtIO MMIO、PCI host 和 initrd；

- 2 从可用物理内存中排除固件、内核、FDT、启动栈和保留区；
- 3 建立最终内核页表和 direct map；
- 4 初始化 buddy page allocator；
- 5 初始化 slab、大对象堆、vmalloc；
- 6 安装 trap、IRQ、时钟源、clockevent 和 timer；
- 7 初始化 VirtIO、块设备、随机数、RTC 和网络接口；
- 8 初始化 VFS、设备节点、`/proc`、`/sys` 和 initramfs；
- 9 挂载 `/dev/vda`，建立 `/mnt/sdcard` 视图；
- 10 初始化任务系统、secondary CPU、IPI 和 workqueue；
- 11 根据命令行或镜像内容选择普通测试、CAgent 或 BuildStorm；
- 12 测试结束后输出完整状态并按比赛路径关机。

这一次序保证页分配器先于堆、trap 先于开中断、调度器先于 secondary CPU 参与运行、块设备先于 sdcard 访问、VFS 先于动态装载器执行。

5. 内存管理设计

5.1 物理内存

启动期使用不依赖堆的 early allocator。FDT 中的 RAM 区间经过对齐、裁剪和保留区扣除后交给 buddy allocator。buddy 按阶管理连续页，页元数据包含状态与引用计数；驱动所需 DMA 区域和普通内存可按约束选择。

5.2 内核堆

内核堆分成两类：

- 小对象走 slab size class，减少频繁页分配和碎片；
- 大对象按连续页申请，在 allocation header 中记录回收信息。

vmalloc 为非连续物理页分配连续内核虚拟区间，并使用 guard page 捕获越界。设备 MMIO 映射复用相同的虚拟地址管理框架，但使用设备属性和不同缓存策略。

5.3 页表与地址空间

- RISC-V 使用 Sv39 三级页表；
- LoongArch 使用架构四级页表和 DMW 直映；
- 公共 RuntimePageTable 提供映射、解除映射、权限修改和地址翻译；
- MappingOptions 执行 $W \wedge X$ 约束；
- 用户地址空间拥有独立根页表和 ASID；
- 每个地址空间记录当前装载它的 CPU mask；
- 页表或后备页释放前必须完成对应 TLB 请求。

5.4 VMA

`VmAreaSet` 保存有序、不重叠的 VMA。它支持 gap 查找、拆分、合并、权限修改、`brk` 扩展、匿名映射和文件映射。为满足 rustc 大量共享对象、线程栈和 allocator arena 同时存在的场景，当前 VMA 上限扩展到 65,536；记录按实际数量在堆上分配，不会在每个进程中预留同等规模的固定数组。

5.5 缺页处理

缺页入口先判断异常来自内核还是用户态，再根据 VMA 权限与后备类型处理：

- 匿名页按需分配并清零；
- 用户栈可在约束范围内向下增长；
- 文件映射从 VFS 对象读取；
- 权限不符返回 fatal fault；
- 内核 copy-to/from-user 将无效访问转换为 `-EFAULT`；
- 页表更新后按范围执行本地或跨核 TLB 失效。

5.6 MAP_FIXED 原子替换

BuildStorm 中的动态链接器和 allocator 会频繁使用 `MAP_FIXED`。实现遵循“先验证、后替换”的原则：

- 1 校验地址对齐、长度、溢出与用户范围；
- 2 计算与目标区间相交的旧 VMA；
- 3 准备新 VMA 与需要退休的页；
- 4 在 MM 锁内完成元数据替换和页表变更；
- 5 发起 TLB 请求；
- 6 TLB 完成后才释放旧页和旧页表页。

`MAP_FIXED_NOREPLACE` 在存在重叠时返回 `EEXIST`，不修改现有映射。该设计避免半更新状态和“页已经释放但远端 CPU TLB 仍指向旧物理页”的竞态。

5.7 共享文件映射写回

最终 BuildStorm 阻塞来自链接器使用可写 `MAP_SHARED | MAP_NORESERVE` 映射构造输出文件。早期实现把文件内容读入匿名页，但解除映射时没有把脏内容写回 VFS。结果是：

- 链接命令返回成功；
- 目标文件长度正确；
- 文件内容仍为零；
- 后续 ELF parser 报告 `Unknown file magic`。

修复后的机制为每个进程登记共享文件映射，记录虚拟区间、文件偏移和文件对象。`munmap` 在拆除页表前遍历受影响页面，把用户映射中的最终字节写回文件，再执行普通退休与 TLB 流程。写回失败会进入失败路径，不会把损坏产物伪装成成功。

此修复同时满足两个关键契约：

- 1 `MAP_SHARED` 的用户写入能被同一文件的后续读取观察到；

- 2 写回发生在页表拆除和后备页释放之前。

6. SMP、调度与并发控制

6.1 CPU 生命周期

CPU 状态区分 discovered、online、active 和 IPI-ready。每个 CPU 拥有：

- idle task；
- run queue；
- 当前任务；
- 当前装载的用户 MM；
- timer 与 reschedule 状态；
- IPI mailbox；
- 独立启动栈和 trap 状态。

secondary CPU 完成 per-CPU 初始化后才发布 active，避免启动 CPU 过早向尚未准备好的目标发送 IPI。

6.2 调度器

当前调度器采用 per-CPU FIFO round-robin，支持：

- timer 抢占；
- `sched_yield`；
- 等待队列与 Completion；
- 任务迁移与 work stealing；
- CPU affinity；
- remote reschedule IPI；
- 退出任务延迟回收；
- 64 KiB guarded kernel stack。

BuildStorm 不是短任务集合。Cargo 会创建并回收大量 rustc 子进程与工作线程，因此调度器必须长期保持 run queue、任务状态和栈所有权一致，不能依赖“任务很快结束”的偶然性。

6.3 MM 切换

每个 CPU 的 `loaded_mm` 明确记录当前用户地址空间。上下文切换在本地中断关闭且持调度器锁时执行：

- 1 验证 outgoing task 与 `loaded_mm` 一致；
- 2 同 MM 切换保留页表；
- 3 不同 MM 切换先恢复内核根页表；
- 4 从旧 MM 的 active mask 移除本 CPU；
- 5 安装并同步新根页表与 ASID；
- 6 将本 CPU 发布到新 MM active mask；

7 最后切换内核栈和寄存器上下文。

6.4 IPI 与 TLB shutdown

IPI mailbox 承载 reschedule、call-function 和 TLB 请求。TLB 请求包含唯一 ID、目标 CPU mask、完成 mask 和失效范围。短范围按页失效，长范围可退化为地址空间级刷新。页表页和物理后备页被放入 retirement batch，只有所有目标 CPU 完成请求后才释放。

6.5 锁与锁序

IrqSpinLock 保存并恢复本地中断状态。lockdep 为锁分配 class 和 rank，检查同 CPU 上的持锁次序。Process、WaitQueue、VM、VFS、allocator 和 TLB 分属不同层级；退出和 fault 等复杂路径会在进入下一层前显式释放上层锁，降低死锁和反向锁序风险。

7. 进程、线程与退出语义

7.1 所有权模型

```
Scheduler Task
+-- guarded KernelStack
+-- Arc<Thread>
    +-- Arc<Process>
        | +-- Arc<UserMm>
        | +-- FileTable
        | +-- SignalState
        | +-- Credentials
        | +-- FsContext
        | +-- children / zombies / process group / session
    +-- TrapFrame / TLS / signal mask
    +-- clear_child_tid / robust list
    +-- scheduler binding / exit state
```

进程的 thread group 只保存线程 ID，不保存 `Arc<Thread>`，避免 Process 与 Thread 形成强引用环。初始线程遵循 `TID == PID`。任务 reaper 在释放退休内核栈和调度器引用后才发布 join completion，保证上层等待者不会观察到仍被调度器使用的对象已经销毁。

7.2 clone 与 execve

clone/clone3 同时支持 fork-like 进程和共享 `CLONE_VM` 的线程路径。实现处理：

- `CLONE_SETTLS`；
- `CLONE_CHILD_SETTID`；
- `CLONE_CHILD_CLEARTID`；
- fd table、signal action、cwd/root 的共享或复制；
- 用户栈和返回值约定；
- 新线程与 scheduler task 的绑定。

execve 构建全新用户 MM，装载 ELF 与解释器，重建初始栈，关闭 `CLOEXEC` fd，并在新镜像准备完成后原子替换旧地址空间。

7.3 退出与等待

线程退出顺序经过长期 BuildStorm 压力修正：

- 1 发布 group-exit 状态；
- 2 停止或唤醒同组线程；
- 3 关闭文件并触发 file-specific `process_exit`；
- 4 使用退出线程自身的 MM 清理 robust list；
- 5 清零 `clear_child_tid` 并 futex wake；
- 6 形成 zombie 状态并唤醒 parent/wait4；
- 7 从 run queue 退休；
- 8 由 reaper 在安全栈上回收内核栈和任务对象。

`wait4` 支持阻塞等待与 `WNOHANG`，并处理信号打断。Cargo 的父子进程树依赖正确的 SIGCHLD、zombie 与 wait 唤醒；任何遗漏都会表现为编译已经结束但外层 cargo 或 shell 永久等待。

7.4 futex 与 robust list

futex key 绑定 MM ASID 和用户地址，防止不同进程相同虚拟地址错误共享队列。实现覆盖常见 wait/wake、bitset、requeue 和超时路径。线程退出时按照 robust-list 链标记 owner-died 并唤醒等待者。该路径对 glibc pthread、rustc 并行任务和动态链接器锁都很重要。

8. Linux ABI 与用户态兼容

8.1 统一 syscall ABI

`kernel/src/syscall.rs` 是系统调用编号和寄存器 ABI 的唯一事实源。两个架构都使用 Linux asm-generic 64 位编号，但寄存器不同：

项目	RISC-V 64	LoongArch 64
syscall number	a7	a7 对应 GPR 11
参数 0-5	a0-a5	GPR 4-9
返回值	a0	GPR 4
指令长度	4 字节	4 字节

errno 以 Linux 负值形式返回。架构层只负责 decode、advance PC 和 set result，具体策略保持共享。

8.2 系统调用覆盖

当前实现覆盖决赛负载实际使用的主要类别：

类别	代表接口
文件与目录	openat、read、write、pread64、pwrite64、getdents64、statx、renameat2
fd 与管道	dup、dup3、fcntl、ioctl、pipe2、sendfile、copy_file_range
内存	brk、mmap、munmap、mremap、mprotect、madvise
进程线程	clone、clone3、execve、exit_group、wait4、set_tid_address

类别	代表接口
同步	futex、set_robust_list、get_robust_list、membarrier、rseq
signal	rt_sigaction、rt_sigprocmask、rt_sigsuspend、rt_sigreturn、tgkill
时间	clock_gettime、clock_nanosleep、gettimeofday、setitimer、times
调度	sched_yield、affinity、scheduler/param、priority、RR interval
epoll/poll	eventfd2、epoll_create1、epoll_ctl、epoll_pwait、ppoll、pselect6
socket	socket、socketpair、bind、listen、accept4、connect、sendmsg、recvmsg
系统信息	uname、sysinfo、prlimit64、getrusage、getrandom、prctl

表中“覆盖”表示存在与当前负载相匹配的实现，不表示所有 Linux flag 和边界条件均已通过完整 LTP。

8.3 ELF 与动态装载

ELF 路径支持：

- ELF64 header 与 program header 校验；
- PT_LOAD 映射和 BSS 清零；
- ET_EXEC 与 ET_DYN；
- PT_INTERP；
- glibc 动态加载器和共享库；
- argc/argv/envp；
- AT_PHDR、AT_PHNUM、AT_ENTRY、AT_BASE、AT_RANDOM、UID/GID、HWCAP、PLATFORM 等 auxv；
- 初始用户栈对齐；
- 文件解释器和 shebang。

BuildStorm 最终产物检查不只依赖大小。诊断路径同时读取内核缓冲区和用户缓冲区的前 16 字节，确认两侧观察到一致的 ELF magic，从而排除 copy-to-user 或再次打开文件时的数据损坏。

9. VFS、文件系统与设备

9.1 VFS

myos-vfs 定义文件对象、操作表、open flags、poll events、stat、dirent 和 errno。内核 fs 层提供：

- tmpfs 根目录；
- fd table 和标准输入输出；

- `cwd/root` 与 `*at` 路径解析；
- `hard link`、`symbolic link`、`rename` 和 `unlink`；
- `mount table`；
- `initramfs newc` 解包；
- `/dev`、`/proc`、`/sys`；
- `ext4 overlay` 与按需物化。

路径解析处理绝对/相对路径、`.`、`..`、`symlink` 和 `dirfd`。文件偏移属于 `open file description`；`dup` 后共享偏移，独立 `open` 不共享。

9.2 ext4 overlay

官方镜像挂载到 `/mnt/sdcard`。为避免一次性复制庞大的 Debian rootfs，内核把常用根目录建立为到 `sdcard` 的别名，并在 `exec/open/stat` 遇到缺失时按需物化对应 `ext4` 目录或文件。

编译期间热点输出使用 `tmpfs`，以减少慢速块设备上的小文件随机写；评分脚本最终要求的精确产物再物化到正式 `target` 路径。选择产物时使用架构和 `release` 目录的确定路径，不再采用全局 `find | head -1`，避免取到旧产物、`binary` 中间文件或另一架构产物。

9.3 pipe、TTY 与 PTY

`pipe` 支持阻塞/非阻塞读写、`EOF`、`EPIPE`、`poll` 和等待唤醒。`TTY` 支持基础 `canonical` 输入、`echo`、退格、`Ctrl-C`、`termios`、`winsize` 与前台进程组。`devpts` 提供 `/dev/ptmx` 与 `/dev/pts/<N>`，满足 `shell` 和部分工具的终端检查。

9.4 procfs 与 sysfs

决赛脚本依赖 `/proc/uptime`、`CPU` 数量、进程和系统信息。`procfs` 的 `uptime` 来自单调时钟，`BuildStorm` 的计时区间由官方脚本读取，内核不修改或伪造。`sysfs` 提供工具和运行库常用的基础目录与属性。

9.5 VirtIO 与块层

`FDT` 或 `PCI host` 用于发现 `VirtIO` 设备。`SudoHal` 向驱动提供 `DMA` 分配和地址转换。块层包含：

- 设备注册表；
- `request queue`；
- 字节范围读写；
- `buffer/page cache`；
- `flush` 与同步；
- `VirtIO block` 后端；
- `/dev/vda`。

`BuildStorm` 暴露了一个重要区别：文件“长度和元数据正确”不等于文件内容已持久。共享 `mmap` 写回修复把 `MM` 与 `VFS` 的一致性边界补齐，避免链接器通过 `mmap` 生成的文件只留下长度而无有效数据。

9.6 网络与随机数

网络层使用 `smoltcp 0.11`，提供 `VirtIO-Net`、`IPv4/IPv6` 接口、`TCP/UDP` 和常用 `socket syscall`。`CAgent` 的 `network` 测试主要检查系统网络状态与基础命令链；当前双架构均已通过。随机数子系统使用 `DRBG`，并可由 `VirtIO RNG` 播种，为 `getrandom` 和 `ELF AT_RANDOM` 提供数据。

10. 决赛执行器

10.1 测试模式选择

运行模式由 `sudoos.oscomp=` 启动参数选择，也可通过镜像中脚本存在性识别。主要模式：

- `final-cagent`；
- `final-buildstorm`；
- `final-buildstorm-diag`；
- `lifecycle stress`；
- `preliminary`/普通 smoke。

生产模式直接运行镜像提供的官方脚本。内核不会自行打印 `BUILDSTORM_COMPILE ok=true`，评分标志只会在官方脚本确认命令返回值、产物路径和产物大小后出现。

10.2 镜像适配原则

生产路径使用 `/mnt/sdcard/glibc/cagent_testcode.sh` 和 `/mnt/sdcard/glibc/buildstorm_testcode.sh`。只要官方镜像保持 `glibc` 目录、`/work/tgskits` 工作区和 Rust 工具链的基本契约，小幅缓存、预编译产物或目录内容差异不会改变评分标志格式，因为最终标志由镜像中的脚本生成。

Bootstrap 对 `tg-xtask` 采用分层策略：

- 1 检查工作区预编译且可执行的精确 `tg-xtask`；
- 2 检查已有 target 中满足条件的候选；
- 3 必要时在 `tmpfs` target 中离线编译；
- 4 将选中的真实二进制安装为 `/tmp/sudoos-buildstorm-bin/tg-xtask`；
- 5 Cargo wrapper 将 `cargo xtask` 路由到该二进制；
- 6 环境准备和 `xtask` 预编译不计入正式编译计时。

11. CAgent 实现与验证

CAgent 包含十类短任务：`factorial`、`date`、`network`、`cpu`、`kernel`、`fs-create`、`fs-readwrite`、`fs-directory`、`fs-search` 和 `fs-usage`。它覆盖 shell、动态链接、时间解析、系统信息、文件创建/读取/遍历和基础网络状态。

11.1 运行准备

- 安装镜像中的动态加载器、`libc`、`bash`、`date` 和必要 `terminfo`；
- 构造稳定 `PATH`；
- 初始化 `/tmp`、`/var/tmp`、`/dev/shm`；
- 直接运行镜像中的官方脚本；
- 对每个 `testcase` 检查 `pass`，任何 `reject` 都视为失败；
- 脚本必须输出 `group end` 并以 0 退出。

11.2 双架构结果

测试	RISC-V 64	LoongArch 64
factorial	1619 ms	1603 ms
date	1963 ms	1645 ms
network	2591 ms	1887 ms
cpu	1758 ms	1434 ms
kernel	1651 ms	1260 ms
fs-create	1536 ms	1059 ms
fs-readwrite	1394 ms	1631 ms
fs-directory	2405 ms	1967 ms
fs-search	2033 ms	1612 ms
fs-usage	1332 ms	1521 ms
结果	10/10 pass	10/10 pass

这些时间来自 2026-08-12 本地最终回归日志。它们用于确认完成性和观察量级；平台时间奖励由平台环境重新计算。

12. BuildStorm 实现

12.1 负载阶段

官方脚本的生产流程为：

- 1 `rustc --version` 与 `cargo --version` ；
- 2 `cargo new` 创建最小项目；
- 3 `cargo build` 编译并运行 Hello World ；
- 4 进入 `/work/tgoskits` ；
- 5 在计时区间外预编译 `tg-xtask` ；
- 6 删除目标架构旧产物；
- 7 读取 `/proc/uptime` ；
- 8 执行 `cargo xtask arceos build -p arceos-helloworld --arch ...` ；
- 9 再次读取 `uptime` ；
- 10 查找产物、计算大小并打印评分记录。

12.2 热点 target 路由

数百 crate 会产生大量短文件、增量元数据和链接器临时输出。为降低 ext4 overlay 压力，正式目标目录在 tmpfs 中完成构建，Cargo wrapper 在完成后将精确 ELF 物化到官方脚本查找的磁盘 target 路径。实现保留以下约束：

- 不修改官方脚本；

- 不改变最终产物内容；
- 不跳过真实编译；
- 不把准备时间计入正式计时；
- 产物必须大于 500 KiB；
- 产物必须能被后续 ELF parser 读取。

12.3 工具链兼容

本地 minibuild 使用 GNU host target, ArceOS formal build 使用对应 musl target。环境中为两架构分别指定 GNU linker, 并为 C/C++ build script 提供按架构选择的交叉编译器 wrapper。CARGO_NET_OFFLINE=true 保证不会因网络状态产生不确定行为。

12.4 长时间运行保障

BuildStorm 生产路径关闭高频诊断打印, 仅保留有界的关键阶段信息。内部统计可记录未知 syscall、futex 操作和 epoll 事件, 但生产状态下不会无限输出。QEMU 外层日志监控同时设置成功和失败正则, 遇到 panic、toolchain fail、minibuild fail 或 compile false 时立即结束。

13. BuildStorm 问题定位与修复过程

13.1 第一阶段：从环境分到真实编译

早期版本只能获得 toolchain/minibuild 环境分。主要问题包括动态 loader 路径、rootfs 物化、用户地址空间容量、Cargo 缓存写入和线程退出后父进程不返回。逐项修复后, 两个架构都进入完整 rustc 编译链。

13.2 第二阶段：多线程退出与等待

编译长期卡住时, 日志显示子进程已经完成用户态工作, 但外层 shell/cargo 没有恢复。修复集中在：

- 退出线程使用自身 MM 执行 robust-list 和 clear-child-tid；
- group-exit 发布顺序；
- SIGCHLD 与 wait4 唤醒；
- futex mismatch 和 timed wait；
- 退出中的任务不能再次进入可运行队列；
- reaper 不能在仍运行的内核栈上释放该栈；
- scheduler CPU identity 与 task 所属 CPU 的一致性。

13.3 第三阶段：VMA 与固定映射

LoongArch rustc 装载 librustc_driver 和 allocator arena 时会超过早期 VMA 容量。扩大容量后, 动态链接器进一步暴露 MAP_FIXED 重叠和退休次序问题。原子替换、PROT_NONE 保留和 MADV_DONTNEED 兼容修复后, 不再出现 Area overlap、SIGSEGV 或 OOM。

13.4 第四阶段：产物路径

tmpfs target 加速后, 官方脚本和 axbuild 仍会访问工作区正式 target 路径。最初使用模糊 find 选择产物, 可能拾取错误候选；随后改为架构、target、profile、文件名均确定的精确路径, 并在物化后校验 source/destination 大小和字节一致性。

13.5 第五阶段：正确大小的全零 ELF

最后一个阻塞最具迷惑性：

```
linker exit = 0
artifact bytes > 1.6 MiB
ELF parser = Unknown file magic
```

通过同时记录 source、destination 和 parser 实际读取文件的前 16 字节，确认文件不是选错，也不是 copy-to-user 失败，而是 mmap 输出从未写回。rust-ldd 先扩展文件长度，再以共享可写映射写入各段；内核只完成了长度变化，没有实现共享映射回写，所以整文件保持零值。

增加共享文件映射登记与 munmap 写回后，双架构均观察到：

```
kernel_magic = 7f 45 4c 46
user_magic  = 7f 45 4c 46
user_copy   = ok
```

随后官方脚本输出 BUILDSTORM_COMPILE mode=multi ok=true。

14. BuildStorm 双架构结果

指标	RISC-V 64	LoongArch 64
vCPU	8	8
内存	8 GiB	8 GiB
toolchain	ok	ok
minibuild	ok	ok
产物	arceos-helloworld	arceos-helloworld
产物大小	1,685,224 B	1,714,736 B
ELF magic	7f454c46	7f454c46
正式计时	3722.00 s	1890.00 s
评分记录	ok=true	ok=true

RISC-V 记录来自 [buildstorm-rv-v3-elf-rv-writeback2-20260811.log](#)，LoongArch 记录来自 [buildstorm-la-v3-elf-la-writeback-20260812.log](#)。两次均完成完整官方脚本主链。

时间成绩不能仅由该表推断。平台 judge 使用与评测机相匹配的 Linux 基线，且镜像中的预编译 tg-xtask、缓存热度和平台资源都会影响总运行时间。当前结果证明编译正确性；平台时间附加分以正式输出为准。

15. 构建、测试与质量门禁

15.1 提交构建

```
make all
```

根 Makefile 的 all 只调用 scripts/oscomp-build.sh，不会启动 QEMU、smoke、stress 或联网任务。构建产物固定为：

```
kernel-rv
kernel-la
```

15.2 本地专项回归

```
make final-cagent-rv
make final-cagent-la
make final-buildstorm-rv
make final-buildstorm-la
```

默认镜像位于外置盘：

```
/Volumes/U/sudoos-final-2026/images/sdcard-rv-pub.img
/Volumes/U/sudoos-final-2026/images/sdcard-la-pub.img
```

可用 `FINAL_IMAGE_RV`、`FINAL_IMAGE_LA`、`FINAL_CPUS`、`FINAL_MEM` 和 `FINAL_RUN_ID` 覆盖。所有 QEMU 运行使用 `-snapshot`，不修改原始镜像。

15.3 成功判据

CAgent：

- `group end` 存在；
- 十个 testcase 各有一条 `pass`；
- 不存在 `reject`；
- 不存在 `panic`；
- 官方脚本退出 0。

BuildStorm：

- `BUILDSTORM_TOOLCHAIN` ok；
- `BUILDSTORM_MINIBUILD` ok；
- 最后一条 `compile` 记录 `mode=multi ok=true`；
- `cores` 与 QEMU 配置一致；
- `elapsed_s` 为正数；
- `bytes` ≥ 500000 ；
- `group end` 存在；
- 不存在 `panic` 或 `compile false`。

15.4 脚本一致性

仓库保留官方 BuildStorm 参考脚本副本和 SHA-256 校验。运行正式 target 前执行 `verify-final-script-sha256`，防止本地参考规则被无意修改。生产内核仍执行镜像中的脚本，副本只用于本地契约校验。

16. 正确性约束清单

以下约束是当前实现最重要的工程边界：

- 1 用户任务运行前，CPU 的 `loaded_mm`、硬件页表根和 MM active mask 必须一致；
- 2 任何用户后备页释放都发生在目标 CPU TLB 完成之后；
- 3 `MAP_FIXED` 失败不得留下半替换 VMA；
- 4 `MAP_SHARED` 文件映射在解除映射前完成数据回写；

- 5 退出清理使用退出线程自身 Process/MM，不依赖可能已经切换的隐式 current；
- 6 退出任务的内核栈只由其他安全上下文回收；
- 7 parent 只能在 child zombie 发布后由 wait4 回收；
- 8 fd close 会执行文件对象的退出钩子，不能只删除表项；
- 9 ELF 成功要求内容、路径、返回码和最低大小同时满足；
- 10 评分标志来自官方脚本，不由内核代写；
- 11 BuildStorm 计时读取真实 [/proc/uptime](#)；
- 12 本地镜像运行使用 snapshot，保证回归之间输入一致。

17. 已知风险与后续工作

17.1 镜像差异

当前实现允许 [tg-xtask](#) 预编译状态、Cargo 缓存内容和部分 target 目录不同。若官方改变以下基础契约，则需要重新核验：

- glibc 脚本路径；
- [/work/tgskits](#) 工作区；
- Rust toolchain 名称和目录；
- logical arch/target 名称；
- 最终产物名称或正式构建命令。

17.2 性能

LoongArch 本地编译时间已经接近公开参考基线量级；RISC-V 仍有明显优化空间。后续应优先用 profile 证据定位，而不是改变评分语义。候选方向包括：

- 减少 VFS 路径解析和 ext4 元数据重复读取；
- 优化 tmpfs 大文件 page lookup；
- 降低全局锁和 run queue 竞争；
- 减少短生命周期进程中的全量地址空间复制；
- 合并 TLB invalidation；
- 降低串口输出和诊断统计开销；
- 检查 RISC-V timer tick、work stealing 与 cache locality。

17.3 可维护性

[kernel/src/user.rs](#) 已经承载大量 syscall 和比赛兼容逻辑。后续宜按 memory、fs、process、signal、socket、poll 和 final runner 拆分，但拆分前需建立稳定的双架构回归，避免纯结构重构引入行为差异。

17.4 长期能力

- 完整 COW fork；
- 统一 page cache 与文件 mmap；

- ext4 journal 与崩溃恢复；
- 更完整的 POSIX signal/job control；
- 网络性能与零拷贝；
- 系统调用级自动差分测试；
- 实体机启动和驱动验证。

18. 复现步骤

18.1 环境

建议准备：

- Rustup 与仓库固定 nightly；
- qemu-system-riscv64；
- qemu-system-loongarch64；
- RISC-V 与 LoongArch 交叉工具链；
- 双架构官方决赛镜像；
- 至少 8 个可用宿主核和 8 GiB 可分配内存。

18.2 构建

```
git checkout main
make all
ls -lh kernel-rv kernel-la
```

18.3 CAgent

```
make final-cagent-rv FINAL_RUN_ID=review-rv
make final-cagent-la FINAL_RUN_ID=review-la
```

检查日志：

```
rg 'testcase cagent .* (pass|reject)|OS COMP TEST GROUP END|script exit' \
artifacts/final-2026/logs/cagent-*.log
```

18.4 BuildStorm

```
make final-buildstorm-rv FINAL_RUN_ID=review-rv
make final-buildstorm-la FINAL_RUN_ID=review-la
```

检查日志：

```
rg 'BUILDSTORM_(TOOLCHAIN|MINIBUILD|COMPILE)|ELF_READ_PROBE|OS COMP TEST GROUP END' \
artifacts/final-2026/logs/buildstorm-*.log
```

必须以最后一条 `BUILDSTORM_COMPILE` 为准，避免早期诊断或失败尝试干扰判断。

19. 文件导航

路径	阅读重点
kernel/src/main.rs	公共初始化、FDT、设备发现、模式选择
arch/riscv64/	RISC-V 启动、SBI、trap、Sv39、SMP
arch/loongarch64/	LoongArch 启动、DMW、trap、页表、SMP
mm/	buddy、slab、VMA、ASID、TLB 与通用地址空间
kernel/src/user_mm.rs	运行期用户页表、fault、fork clone、退休批次
kernel/src/process.rs	Process/Thread 所有权、fd、signal、退出状态
kernel/src/task/	scheduler、wait queue、completion、stack、reaper
kernel/src/syscall.rs	双架构 asm-generic ABI
kernel/src/user.rs	syscall 实现、用户执行链、CAgent/BuildStorm runner
kernel/src/elf.rs	ELF 元数据校验
kernel/src/exec.rs	ELF 映射、解释器、auxv、初始用户栈
kernel/src/fs/	VFS 集成、路径、tmpfs、mount 与物化
kernel/src/ext4.rs	ext4 访问与 overlay
kernel/src/net/	VirtIO-Net、smoltcp 与 socket
scripts/oscomp-build.sh	正式双架构构建入口
scripts/regress-final-beta1.sh	本地评测记录解析与严格判据

20. 结论

SudoOS-Plus 当前版本已经从“能启动和运行基础用户程序”推进到“能在两个架构上承载真实 Rust 工具链和复杂并行构建”。项目的关键进展不只在系统调用数量，而在多个子系统之间的语义闭环：调度器与 MM 的装载关系、TLB 与页回收次序、线程退出与 wait/futex 唤醒、动态链接与 VMA、共享 mmap 与 VFS 写回，以及官方脚本与真实产物之间的验证链。

CAgent 双架构十项通过，说明 shell、动态链接、时间、文件系统和基础系统信息路径稳定；BuildStorm 双架构生成有效 ELF 并输出 `ok=true`，说明内核能够持续承载 Cargo、rustc、链接器和高并发文件操作。最终平台成绩仍需由官网镜像和 judge 给出，但当前实现已经具备可复现、可审计、可继续优化的技术基础。

附录 A：本地验证记录

A.1 RISC-V BuildStorm

```
BUILDSTORM_TOOLCHAIN ok
BUILDSTORM_MINIBUILD ok
ELF_READ_PROBE ... kernel_magic=[7f, 45, 4c, 46, ...]
    user_magic=[7f, 45, 4c, 46, ...] user_copy=ok
BUILDSTORM_COMPILE mode=multi ok=true elapsed_s=3722.00 cores=8
    bytes=1685224 arch=riscv64
#### OS COMP TEST GROUP END buildstorm-glibc ####
```

A.2 LoongArch BuildStorm

```
BUILDSTORM_TOOLCHAIN ok
BUILDSTORM_MINIBUILD ok
ELF_READ_PROBE ... kernel_magic=[7f, 45, 4c, 46, ...]
    user_magic=[7f, 45, 4c, 46, ...] user_copy=ok
BUILDSTORM_COMPILE mode=multi ok=true elapsed_s=1890.00 cores=8
    bytes=1714736 arch=loongarch64
#### OS COMP TEST GROUP END buildstorm-glibc ####
```

A.3 CAgent

RISC-V 64 : 10 pass, 0 reject, script exit=0
LoongArch 64: 10 pass, 0 reject, script exit=0

附录 B：参考资料

资料	地址或说明
OSCOMP 2026 内核赛道线上决赛测例	https://github.com/oscomp/testsuite-for-oskernel/tree/final-2026
Linux asm-generic syscall ABI	https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/include/uapi/asm-generic/unistd.h
RISC-V Privileged Architecture Specification	https://riscv.org/technical/specifications/
LoongArch Reference Manual	https://loongson.github.io/LoongArch-Documentation/
VirtIO Specification	https://docs.oasis-open.org/virtio/virtio/
Rust ELF、Cargo 与目标平台	以仓库固定工具链和官方镜像内版本为准